



Red Hat build of Apache Camel 4.10

Migrating to Red Hat build of Apache Camel for Spring Boot

Migrating to Red Hat build of Apache Camel for Spring Boot

Red Hat build of Apache Camel 4.10 Migrating to Red Hat build of Apache Camel for Spring Boot

Migrating to Red Hat build of Apache Camel for Spring Boot

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

This guide describes the settings for Red Hat build of Apache Camel components.

Table of Contents

CHAPTER 1. ABOUT THE MIGRATION GUIDE	4
CHAPTER 2. OVERVIEW OF MIGRATING RED HAT FUSE 7 APPLICATIONS TO RED HAT BUILD OF APACHE CAMEL FOR SPRING BOOT	5
2.1. FUSE	5
2.2. RED HAT BUILD OF APACHE CAMEL FOR SPRING BOOT	5
2.3. CAMEL ON EAP	5
2.4. STANDARD MIGRATION PATHS	5
2.4.1. XML Path	6
2.4.2. Java Path	6
2.4.3. Architectural changes	6
2.4.4. The javax to jakarta Package Namespace Change	6
2.4.5. Migration tools	7
2.5. MIGRATING THE APPLICATION TO CAMEL SPRING BOOT	7
2.5.1. Migrating the pom.xml file	7
2.6. JAVA DSL ROUTE MIGRATION EXAMPLE	7
CHAPTER 3. MIGRATING TO APACHE CAMEL 4	9
3.1. JAVA VERSIONS	9
3.2. REMOVED COMPONENTS	9
3.3. LOGGING	10
3.4. JUNIT 4	11
3.5. USING STREAM CACHE AND SPOOLING	11
3.6. API CHANGES	11
3.7. EIP CHANGES	12
3.7.1. Poll Enrich EIP	12
3.7.2. CircuitBreaker EIP	12
3.8. XML DSL	13
3.9. TYPE CONVERTER	14
3.10. TRACING	14
3.11. USEORIGINALMESSAGE / USEORIGINALBODY	14
3.12. CAMEL HEALTH	14
3.12.1. Producer Health Checks	14
3.13. JMX	15
3.14. YAML DSL	15
3.15. BACKLOG TRACING	15
3.16. XML SERIALIZATION	16
3.17. OPENAPI MAVEN PLUGIN	16
3.18. COMPONENT CHANGES	16
3.18.1. Category	16
3.18.2. camel-openapi-rest-dsl-generator	16
3.18.3. camel-atom	16
3.18.4. camel-azure-cosmosdb	16
3.18.5. camel-bean	16
3.18.6. camel-box	17
3.18.7. camel-caffeine	17
3.18.8. camel-fhir	17
3.18.9. camel-google	17
3.18.10. camel-http	17
3.18.11. camel-http-common	17
3.18.12. camel-kubernetes	17
3.18.13. camel-main	18

3.18.14. camel-micrometer	18
3.18.15. camel-jbang	19
3.18.16. camel-openapi-java	19
3.18.17. camel-salesforce	19
3.18.18. camel-slack	19
3.18.19. camel-spring-rabbitmq	19
3.19. CAMEL SPRING BOOT	19
3.19.1. Using restConfiguration in a Spring Bean XML	19
3.19.2. Graceful Shutdown	20
3.19.3. camel-micrometer-starter	20
3.19.4. camel-platform-http-starter	20
3.19.5. camel-twitter	20
CHAPTER 4. MIGRATING TO APACHE CAMEL 3	22
4.1. MODULARIZATION OF CAMEL-CORE	22
4.2. MODULARIZATION OF COMPONENTS	23
4.3. DEFAULT SHUTDOWN STRATEGY	24
4.4. MULTIPLE CAMELCONTEXTS PER APPLICATION NOT SUPPORTED	24
4.5. DEPRECATED APIS AND COMPONENTS	24
4.5.1. Removed components	25
4.5.2. Renamed components	25
4.6. CHANGES TO CAMEL COMPONENTS	26
4.6.1. Mock component	26
4.6.2. ActiveMQ	26
4.6.3. AWS	26
4.6.4. Camel CXF	27
4.6.4.1. Camel CXF changed namespaces	27
4.6.5. FHIR	28
4.6.6. Kafka	28
4.6.7. Telegram	28
4.6.8. JMX	28
4.6.9. XSLT	28
4.6.10. XML DSL Migration	28
4.7. MIGRATING CAMEL MAVEN PLUGINS	29
CHAPTER 5. UPGRADING RED HAT BUILD OF APACHE CAMEL ON SPRING BOOT VERSION WITH CAMEL UPDATE RECIPES	30
5.1. RUNNING CAMEL UPDATES	30
5.1.1. Configuration Options	30
5.1.2. Updating a plain Camel application	31
5.1.3. Updating a Spring Boot application	31

CHAPTER 1. ABOUT THE MIGRATION GUIDE

This guide details the changes in the Apache Camel components that you must consider when migrating your application. This guide provides information about following changes.

- Supported Java versions
- Changes to Apache Camel components and deprecated components
- Changes to APIs and deprecated APIs
- Updates to EIP
- Updated to tracing and health checks

CHAPTER 2. OVERVIEW OF MIGRATING RED HAT FUSE 7 APPLICATIONS TO RED HAT BUILD OF APACHE CAMEL FOR SPRING BOOT

2.1. FUSE

Red Hat Fuse is an agile integration solution based on open source communities like Apache Camel and Apache Karaf. Red Hat Fuse is a lightweight, flexible integration platform that enables rapid on-premise cloud integration. You can run Red Hat Fuse using three different runtimes:

- Karaf which supports OSGi applications
- Spring Boot
- JBoss EAP (Enterprise Application Platform)

2.2. RED HAT BUILD OF APACHE CAMEL FOR SPRING BOOT

Red Hat build of Apache Camel for Spring Boot provides auto-configuration of the starters for many Camel components. The opinionated auto-configuration of the Camel context auto-detects Camel routes available in the Spring context and registers the key Camel utilities (like producer template, consumer template and the type converter) as beans. Camel Spring Boot takes advantage of the many performance improvements made in Apache Camel, which results in a lower memory footprint, less reliance on reflection, and faster startup times.

In a Red Hat build of Apache Camel for Spring Boot application, you can define Camel routes using Java DSL, so you can migrate the Camel routes that you use in your Fuse application to Camel Spring Boot.

2.3. CAMEL ON EAP

Karaf, which follows the OSGi dependency management concept, and EAP, which follows the JEE specification, are application servers impacted by the adoption of containerized applications. Containers have emerged as the predominant method for packaging applications. Consequently, the responsibility for managing applications, which encompasses deployment, scaling, clustering, and load balancing, has shifted from the application server to the container orchestration using Kubernetes.

Although EAP continues to be supported on Red Hat Openshift, Camel 3 is no longer supported on an EAP server. So if you have a Fuse 7 application running on an EAP server, you should consider migrating your application to the Red Hat Build of Apache Camel for Spring Boot or the Red Hat build of Apache Camel for Quarkus. The benefit of the migration process is to consider a redesign, or partial redesign of your application, from a monolith to a microservices architecture.

If you do not use Openshift, RHEL virtual machines remain a valid approach when you deploy your application for Spring Boot and Quarkus, and Quarkus also benefits from its native compilation capabilities. It is important to evaluate the tooling to support the management of a microservices architecture on such a platform. Red Hat provides this capability through Ansible, using the Red Hat Ansible for Middleware collections.

Openshift has replaced Fabric8 as the runtime platform for Fuse 6 users and is the recommended target for your Fuse application migration.

2.4. STANDARD MIGRATION PATHS

Following section describes the standard migration paths that you can consider before migrating your applications to Red Hat build of Apache Camel for Spring Boot.

2.4.1. XML Path

Fuse applications written in Spring XML or Blueprint XML should be migrated towards an XML-based flavor, and can target either the Spring Boot or the Quarkus runtime with no difference in the migration steps.

2.4.2. Java Path

Fuse applications written in Java DSL should be migrated towards a Java-based flavor, and can target either the Spring Boot or the Quarkus runtime with no difference in the migration steps.

2.4.3. Architectural changes

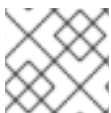
Consider the following architectural changes when you are migrating your application:

- If your Fuse 6 application relied on the Fabric8 service discovery, you should use Kubernetes Service Discovery when running Camel 3 on OpenShift.
- If your Fuse 6 application relies on OSGi bundle configuration, you should use Kubernetes ConfigMaps and Secrets when running Camel 3 on OpenShift.
- If your application uses a file-based route definition, consider using AWS S3 technology when running Camel 3 on OpenShift.
- If your application uses a standard filesystem, the resulting Spring Boot or Quarkus applications should be deployed on standard RHEL virtual machines rather than the Openshift platform.
- Delegation of Inbound HTTPS connections to the Openshift Router which handles SSL requirements.
- Delegation of Hystrix features to Service Mesh.

2.4.4. The javax to jakarta Package Namespace Change

The Java EE move to the Eclipse Foundation and the establishment of Jakarta EE, since Jakarta EE 9, packages used for all EE APIs have changed to **jakarta.***

Code snippets in documentation have been updated to use the **jakarta.*** namespace, but you of course need to take care and review your own applications.



NOTE

This change does not affect javax packages that are part of Java SE.

When migrating applications to EE 10, you need to:

- Update any import statements or other source code uses of EE API classes from the **javax** package to **jakarta**.
- Change any EE-specified system properties or other configuration properties whose names begin with **javax.** to begin with **jakarta..**

- Use the **META-INF/services/jakarta.[rest_of_name]** name format to identify implementation classes in your applications that use the implement EE interfaces or abstract classes bootstrapped with the **java.util.ServiceLoader** mechanism.

2.4.5. Migration tools

- Source code migration: [How to use Red Hat Migration Toolkit for Auto-Migration of an Application to the Jakarta EE 10 Namespace](#)
- Bytecode transforms: For cases where source code migration is not an option, the open source [Eclipse Transformer](#)

Additional resources

- Background: [Update on Jakarta EE Rights to Java Trademarks](#)
- Red Hat Customer Portal: [Red Hat JBoss EAP Application Migration from Jakarta EE 8 to EE 10](#)
- Jakarta EE: [Javax to Jakarta Namespace Ecosystem Progress](#)

2.5. MIGRATING THE APPLICATION TO CAMEL SPRING BOOT

Apache Camel is a versatile integration framework that enables seamless communication and data exchange between different applications. Key components of Apache Camel on Spring Boot include endpoints, processors, and routes. Configuration options in Spring Boot allow you to fine-tune the behavior of Apache Camel, making it a flexible and customizable integration solution.

2.5.1. Migrating the pom.xml file

Update the existing project's **pom.xml** file. In the original Spring Boot application, the **pom.xml** contained dependencies related to Spring Boot and Apache Camel. Update the dependencies so that the application will run using the latest Camel Spring Boot.

2.6. JAVA DSL ROUTE MIGRATION EXAMPLE

To migrate a Java DSL route definition from your Fuse application to CSB, you can copy your existing route definition directly to your CSB application and add the necessary dependencies to your application's **pom.xml** file.

In this example, we will migrate a content-based route definition from a Fuse 7 application to a new CSB application by copying the Java DSL route to a file named **MyCamelRouter.java** in your CSB application.

Procedure

1. Download the [Spring Boot example](#). This example shows how to work with a simple Apache Camel application using Spring Boot.
2. Navigate to the directory where you extracted the example files from the previous step:

```
$ cd <directory_name>
```

3. Locate a file named **MyCamelRouter.java** in the **src/main/java/sample/camel/** subfolder.

4. Add the route definition from your Fuse application to the **MyCamelRouter.java**, similar to the following example:

```
package sample.camel;

import org.apache.camel.builder.RouteBuilder;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

/**
 * A simple Camel route that triggers from a timer and calls a bean and prints to system out.
 * <p/>
 * Use <tt>@Component</tt> to make Camel auto-detect this route when starting.
 */
@Component
public class MyCamelRouter extends RouteBuilder {

    // we can use spring dependency injection
    @Autowired
    MyBean myBean;

    @Override
    public void configure() throws Exception {
        // start from a timer
        from("timer:hello?period={{myPeriod}}").routeId("hello")
            // and call the bean
            .bean(myBean, "saySomething")
            // and print it to system out via stream component
            .to("stream:out");
    }
}
```

5. Run your CSB application.

```
mvn spring-boot:run
```

CHAPTER 3. MIGRATING TO APACHE CAMEL 4

This section provides information that can help you migrate your Apache Camel applications from version 3.20 or higher to 4.0. If you are upgrading from an older Camel 3.x release, such as 3.14, see the individual [Upgrade guide](#) to upgrade to the 3.20 release, before upgrading to Apache Camel 4.

NOTE

The information in the Migration guide is not applicable for IBM Power and IBM Z for this release. This is expected to change in future releases.

3.1. JAVA VERSIONS

Apache Camel 4 supports Java 17. Support for Java 11 is dropped.

3.2. REMOVED COMPONENTS

The following components has been removed:

Component	Alternative component(s)
camel-any23	none
camel-atlasmap	none
camel-atmos	none
camel-caffeine-lrucache	camel-cache, camel-ignite, camel-infinispan
camel-cdi	camel-spring-boot, camel-quarkus
camel-corda	none
camel-directvm	camel-direct
camel-dozer	camel-mapstruct
camel-elasticsearch-rest	camel-elasticsearch
camel-gora	none
camel-hbase	none
camel-hyperledger-aries	none
camel-iota	none
camel-ipfs	none

Component	Alternative component(s)
camel-jbpm	none
camel-jclouds	none
camel-johnzon	camel-jackson, camel-fastjson, camel-gson
camel-microprofile-metrics	camel-micrometer, camel-opentelemetry
camel-milo	none
camel-opentracing	camel-micrometer, camel-opentelemetry
camel-rabbitmq	spring-rabbitmq-component
camel-rest-swagger	camel-openapi-rest
camel-restdsl-swagger-plugin	camel-restdsl-openapi-plugin
camel-resteasy	camel-cxf, camel-rest
camel-solr	none
camel-spark	none
camel-spring-integration	none
camel-swagger-java	camel-openapi-java
camel-websocket	camel-vertx-websocket
camel-websocket-jsr356	camel-vertx-websocket
camel-vertx-kafka	camel-kafka
camel-vm	camel-seda
camel-weka	none
camel-xstream	camel-jacksonxml
camel-zipkin	camel-micrometer, camel-opentelemetry

3.3. LOGGING

Camel 4 has upgraded logging facade API **slf4j-api** from 1.7 to 2.0.

3.4. JUNIT 4

All the **camel-test** modules that were JUnit 4.x based has been removed. All test modules now use JUnit 5.

3.5. USING STREAM CACHE AND SPOOLING

Streams are cached in memory. When you are migrating the applications from Red Hat Fuse to Red Hat build of Apache Camel for Spring Boot, spooling must be enabled to prevent the Out Of Memory Errors with big files. For large stream messages, you can set the **spoolEnabled=true** and then large message (over 128 KB) will be cached in a temporary file instead. Camel itself will handle deleting the temporary file once the cached stream is no longer necessary. In Camel 3 and above the Stream Cache is enabled by default, but the spool is disabled and needs to be enabled. You can use the following property for Camel Spring Boot.

```
camel.springboot.stream-caching-spool-enabled=true
```

3.6. API CHANGES

Following APIs are deprecated and removed from version 4:

- The **org.apache.camel.ExchangePattern** has removed **InOptionalOut**.
- Removed **getEndpointMap()** method from **CamelContext**.
- Removed **@FallbackConverter** as you should use **@Converter(fallback = true)** instead.
- Removed **uri** attribute on **@EndpointInject**, **@Produce**, and **@Consume** as you should use **value** (default) instead. For example **@Produce(uri = "kafka:cheese")** should be changed to **@Produce("kafka:cheese")**
- Removed **label** on **@UriEndpoint** as you should use **category** instead.
- Removed all **asyncCallback** methods on **ProducerTemplate**. Use **asyncSend** or **asyncRequest** instead.
- Removed **org.apache.camel.spi.OnCamelContextStart**. Use **org.apache.camel.spi.OnCamelContextStarting** instead.
- Removed **org.apache.camel.spi.OnCamelContextStop**. Use **org.apache.camel.spi.OnCamelContextStopping** instead.
- Decoupled the **org.apache.camel.ExtendedCamelContext** from the **org.apache.camel.CamelContext**.
- Replaced **adapt()** from **org.apache.camel.CamelContext** with **getCamelContextExtension**
- Decoupled the **org.apache.camel.ExtendedExchange** from the **org.apache.camel.Exchange**.
- Replaced **adapt()** from **org.apache.camel.ExtendedExchange** with **getExchangeExtension**

- Exchange failure handling status has moved from being a property defined as **ExchangePropertyKey.FAILURE_HANDLED** to a member of the `ExtendedExchange`, accessible via `isFailureHandled()` method.
- Removed **Discard** and **DiscardOldest** from **`org.apache.camel.util.concurrent.ThreadPoolRejectedPolicy`**.
- Removed **`org.apache.camel.builder.SimpleBuilder`**. Was mostly used internally in Camel with the Java DSL in some situations.
- Moved **`org.apache.camel.support.IntrospectionSupport`** to **`camel-core-engine`** for internal use only. End users should use **`org.apache.camel.spi.BeanInspection`** instead.
- Removed **`archetypeCatalogAsXml`** method from **`org.apache.camel.catalog.CamelCatalog`**.
- The **`org.apache.camel.health.HealthCheck`** method **`isLiveness`** is now default **`false`** instead of **`true`**.
- Added **`position`** method to **`org.apache.camel.StreamCache`**.
- The method **`configure`** from the interface **`org.apache.camel.main.Listener`** was removed
- The **`org.apache.camel.support.EventNotifierSupport`** abstract class now implements **`CamelContextAware`**.
- The type for **`dumpRoutes`** on **`CamelContext`** has changed from **`boolean`** to **`String`** to allow specifying either xml or yaml.



NOTE

The **`org.apache.camel.support.PluginHelper`** gives easy access to various extensions and context plugins, that was available previously in Camel v3 directly from **`CamelContext`**.

3.7. EIP CHANGES

- Removed **`lang`** attribute for the **`<description>`** on every EIPs.
- The **`InOnly`** and **`InOut`** EIPs has been removed. Instead, use **`SetExchangePattern`** or **`To`** where you can specify exchange pattern to use.

3.7.1. Poll Enrich EIP

The polled endpoint URI is now stored as property on the **`Exchange`** (with key **`CamelToEndpoint`**) like all other EIPs. Before the URI was stored as a message header.

3.7.2. CircuitBreaker EIP

The following options in **`camel-resilience4j`** was mistakenly not defined as attributes:

Option

bulkheadEnabled
bulkheadMaxConcurrentCalls
bulkheadMaxWaitDuration
timeoutEnabled
timeoutExecutorService
timeoutDuration
timeoutCancelRunningFuture

These options were not exposed in YAML DSL, and in XML DSL you need to migrate from:

```
<circuitBreaker>
  <resilience4jConfiguration>
    <timeoutEnabled>true</timeoutEnabled>
    <timeoutDuration>2000</timeoutDuration>
  </resilience4jConfiguration>
  ...
</circuitBreaker>
```

To use following attributes instead:

```
<circuitBreaker>
  <resilience4jConfiguration timeoutEnabled="true" timeoutDuration="2000"/>
  ...
</circuitBreaker>
```

3.8. XML DSL

The **<description>** to set a description on a route or node, has been changed from an element to an attribute.

Example

Changed from

```
<route id="myRoute">
  <description>Something that this route do</description>
  <from uri="kafka:cheese"/>
  ...
</route>
```

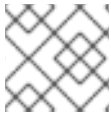
To

```
<route id="myRoute" description="Something that this route do">
  <from uri="kafka:cheese"/>
  ...
```

```
</route>
```

We recommend that you use **xml-io-dsl** parser for Camel XML DSL. It works with all Camel runtimes. To use the **xml-io-dsl** parser, add the following dependency to your project's **pom.xml**.

```
<dependency>
  <groupId>org.apache.camel.springboot</groupId>
  <artifactId>camel-xml-io-dsl-starter</artifactId>
  <version>{CamelSBProjectVersion}</version>
</dependency>
```



NOTE

The **version** can be omitted if you are using the BOM for your application.

For more information about XML IO DSL, refer [XML IO DSL](#).

3.9. TYPE CONVERTER

The **String** → **java.io.File** converter has been removed.

3.10. TRACING

The [Tracer](#) and [Backlog Tracer](#) no longer includes internal tracing events from routes that was created by Rest DSL or route templates or Kamelets. You can turn this on, by setting **traceTemplates=true** in the tracer.

The [Backlog Tracer](#) has been enhanced and *fixed* to trace message headers (also streaming types). This means that previously headers of type **InputStream** was not traced before, but is now included. This could mean that the header stream is positioned at end, and logging the header afterward, may appear as the header value is empty.

3.11. USEORIGINALMESSAGE / USEORIGINALBODY

When **useOriginalMessage** or **useOriginalBody** is enabled in **OnException**, **OnCompletion** or error handlers, then the original message body is defensively copied and if possible converted to **StreamCache** to ensure the body can be re-read when accessed. Previously the original body was not converted to **StreamCache** which could lead to the body not able to be read or the stream has been closed.

3.12. CAMEL HEALTH

Health checks are now by default only readiness checks out of the box. Camel provides the **CamelContextCheck** as both readiness and liveness checks, so there is at least one of each out of the box. Only consumer based health-checks is enabled by default.

3.12.1. Producer Health Checks

The option **camel.health.components-enabled** has been renamed to **camel.health.producers-enabled**.

Some components (in particular AWS) provides also health checks for producers; in Camel 3.x these health checks did not work properly and has been disabled in the source. To continue this behaviour in Camel 4, the producer based health checks are disabled.

Notice that **camel-kafka** comes with producer based health-check that worked in Camel 3, and therefore this change in Camel 4, means that this health-check is disabled.

You **MUST** enable producer health-checks globally, such as in **application.properties**:

```
camel.health.producers-enabled = true
```

3.13. JMX

Camel now also include MBeans for **doCatch** and **doFinally** in the tree of processor MBeans.

The **ManagedChoiceMBean** have renamed **choiceStatistics** to **extendedInformation**. The **ManagedFailoverLoadBalancerMBean** have renamed **exceptionStatistics** to **extendedInformation**.

The **CamelContextMBean** and **CamelRouteMBean** has removed method **dumpRouteAsXml(boolean resolvePlaceholders, boolean resolveDelegateEndpoints)**.

3.14. YAML DSL

The backwards compatible mode Camel 3.14 or older, which allowed to have *steps* as child to *route* has been removed.

The old syntax:

```
- route:
  from:
    uri: "direct:info"
  steps:
    - log: "message"
```

should be changed to:

```
- route:
  from:
    uri: "direct:info"
  steps:
    - log: "message"
```

3.15. BACKLOG TRACING

The option **backlogTracing=true** is now automatically enabled to start the tracer on startup. In the previous versions the tracer was only made available, and had to be manually enabled afterwards. The old behavior can be archived by setting **backlogTracingStandby=true**.

Move the following class from

org.apache.camel.api.management.mbean.BacklogTracerEventMessage in **camel-management-api** JAR to **org.apache.camel.spi.BacklogTracerEventMessage** in **camel-api** JAR.

The **org.apache.camel.impl.debugger.DefaultBacklogTracerEventMessage** has been refactored into an interface **org.apache.camel.spi.BacklogTracerEventMessage** with some additional details about traced messages. For example Camel now captures a *first* and *last* trace that contains the input and outgoing (if **InOut**) messages.

3.16. XML SERIALIZATION

The default xml serialization using **ModelToXMLDumper** has been improved and now uses a generated xml serializer located in the **camel-xml-io** module instead of the JAXB based one from **camel-jaxb**.

3.17. OPENAPI MAVEN PLUGIN

The **camel-restdsl-openapi-plugin** Maven plugin now uses **platform-http** as the default rest component in the generated Rest DSL code. Previously the default was servlet. However, platform-http is a better default that works out of the box with Spring Boot and Quarkus.

3.18. COMPONENT CHANGES

3.18.1. Category

The number of enums for **org.apache.camel.Category** has been reduced from 83 to 37, which means custom components that are using removed values need to choose one of the remainder values. We have done this to consolidate the number of categories of all components in the Camel community.

3.18.2. camel-openapi-rest-dsl-generator

This dsl-generator has updated the underlying model classes (**apicurio-data-models**) from 1.1.27 to 2.0.3.

3.18.3. camel-atom

The **camel-atom** component has changed the 3rd party atom client from Apache Abdera to RSSReader. This means the feed object is changed from **org.apache.abdera.model.Feed** to **com.apptasticsoftware.rssreader.Item**.

3.18.4. camel-azure-cosmosdb

The **itemPartitionKey** has been updated. It's now a String a not a PartitionKey anymore. More details in CAMEL-19222.

3.18.5. camel-bean

When using the **method** option to refer to a specific method, and using parameter types and values, such as: **"bean:myBean?method=foo(com.foo.MyOrder, true)"** then any class types must now be using **.class** syntax, i.e. **com.foo.MyOrder** should now be **com.foo.MyOrder.class**.

Example

```
"bean:myBean?method=foo(com.foo.MyOrder.class, true)"
```

This also applies to Java types such as String, int.

```
"bean:myBean?method=bar(String.class, int.class)"
```

3.18.6. camel-box

Upgraded from Box Java SDK v2 to v4, which have some method signature changes. The method to get a file thumbnail is no longer available.

3.18.7. camel-caffeine

The **keyType** parameter has been removed. The Key for the cache will now be only **String** type. More information in CAMEL-18877.

3.18.8. camel-fhir

The underlying **hapi-fhir** library has been upgraded from 4.2.0 to 6.2.4. Only the **Delete** API method has changed and now returns **ca.uhn.fhir.rest.api.MethodOutcome** instead of **org.hl7.fhir.instance.model.api.IBaseOperationOutcome**. See [hapi-fhir](#) for a more detailed list of underlying changes (only the hapi-fhir client is used in Camel).

3.18.9. camel-google

The API based components **camel-google-drive**, **camel-google-calendar**, **camel-google-sheets** and **camel-google-mail** has been upgraded from Google Java SDK v1 to v2 and to latest API revisions. The **camel-google-drive** and **camel-google-sheets** have some API methods changes, but the others are identical as before.

3.18.10. camel-http

The component has been upgraded to use Apache HttpComponents v5 which has an impact on how the underlying client is configured. There are 4 different timeouts (**connectionRequestTimeout**, **connectTimeout**, **soTimeout** and **responseTimeout**) instead of initially 3 (**connectionRequestTimeout**, **connectTimeout** and **socketTimeout**) and the default value of some of them has changed so please refer to the documentation for more details.

Please note that the **socketTimeout** has been removed from the possible configuration parameters of **HttpClient**, use **responseTimeout** instead.

Finally, the option **soTimeout** along with any parameters included into **SocketConfig**, need to be prefixed by **httpConnection.**, the rest of the parameters including those defined into **HttpClientBuilder** and **RequestConfig** still need to be prefixed by **httpClient.** like before.

3.18.11. camel-http-common

The API in **org.apache.camel.http.common.HttpBinding** has changed slightly to be more reusable. The **parseBody** method now takes in **HttpServletRequest** as input parameter. And all **HttpMessage** has been changed to generic **Message** types.

3.18.12. camel-kubernetes

The **io.fabric8:kubernetes-client** library has been upgraded and some deprecated API usage has been removed. Operations previously prefixed with **replace** are now prefixed with **update**.

For example **replaceConfigMap** is now **updateConfigMap**, **replacePod** is now **updatePod** etc. The corresponding constants in class **KubernetesOperations** are also renamed.

REPLACE_CONFIGMAP_OPERATION is now **UPDATE_CONFIGMAP_OPERATION**,
REPLACE_POD_OPERATION is now **UPDATE_POD_OPERATION** etc.

3.18.13. camel-main

The following constants has been moved from **BaseMainSupport** / **Main** to **MainConstants**:

Old Name	New Name
Main.DEFAULT_PROPERTY_PLACEHOLDER_LOCATION	MainConstants.DEFAULT_PROPERTY_PLACEHOLDER_LOCATION
Main.INITIAL_PROPERTIES_LOCATION	MainConstants.INITIAL_PROPERTIES_LOCATION
Main.OVERRIDE_PROPERTIES_LOCATION	MainConstants.OVERRIDE_PROPERTIES_LOCATION
Main.PROPERTY_PLACEHOLDER_LOCATION	MainConstants.PROPERTY_PLACEHOLDER_LOCATION

3.18.14. camel-micrometer

The metrics has been renamed to follow Micrometer naming convention .

Old Name	New Name
CamelExchangeEventNotifier	camel.exchange.event.notifier
CamelExchangesFailed	camel.exchanges.failed
CamelExchangesFailuresHandled	camel.exchanges.failures.handled
CamelExchangesInflight	camel.exchanges.external.redeliveries
CamelExchangesSucceeded	camel.exchanges.succeeded
CamelExchangesTotal	camel.exchanges.total
CamelMessageHistory	camel.message.history
CamelRoutePolicy	camel.route.policy
CamelRoutePolicyLongTask	camel.route.policy.long.task
CamelRoutesAdded	camel.routes.added

CamelRoutesRunning	camel.routes.running
--------------------	----------------------

3.18.15. camel-jbang

The command **camel dependencies** has been renamed to **camel dependency**.

In Camel CLI the **-dir** parameter for **init** and **run** goal has been renamed to require 2 dashes **--dir** like all the other options.

The **camel stop** command will now by default stop all running integrations (the option **--all** has been removed).

The *Placeholders substitutes* is changed to use **#name** instead of **\$name** syntax.

3.18.16. camel-openapi-java

The **camel-openapi-java** component has been changed to use **io.swagger.v3** libraries instead of **io.apicurio.datamodels**. As a result, the return type of the public method `org.apache.camel.openapi.RestOpenApiReader.read()` is now **io.swagger.v3.oas.models.OpenAPI** instead of **io.apicurio.datamodels.openapi.models.OasDocument**. When an OpenAPI 2.0 (swagger) specification is parsed, it is automatically upgraded to OpenAPI 3.0.x by the swagger parser. This version also supports OpenAPI 3.1.x specifications. The related spring-boot starter components have been modified to use the new return type.

3.18.17. camel-salesforce

Property names of blob fields on generated DTOs no longer have 'Url' affixed. For example, the **ContentVersionUrl** property is now **ContentVersion**.

3.18.18. camel-slack

The default delay (on slack consumer) is changed from 0.5s to 10s to avoid being rate limited to often by Slack.

3.18.19. camel-spring-rabbitmq

The option **replyTimeout** in **camel-spring-rabbitmq** has been fixed and the default value from 5 to 30 seconds (this is the default used by Spring).

3.19. CAMEL SPRING BOOT

The **camel-spring-boot** dependency no longer includes **camel-spring-xml**. To use legacy Spring XML files **<beans>** with Camel on Spring Boot, then include the **camel-spring-boot-xml-starter** dependency.

3.19.1. Using restConfiguration in a Spring Bean XML

When declaring a **restConfiguration** in a Spring Beans XML file, for example:

```
<camelContext id="SampleCamel" xmlns="http://camel.apache.org/schema/spring">
  <restConfiguration port="8080" bindingMode="off" scheme="http">
```

```

        contextPath="myapi" apiContextPath="myapi/swagger">
    </restConfiguration>

```

The Camel Spring Boot application does not start due to a circular references error. The **RestConfigurationDefinitionAutoConfiguration** property is used when you use the Spring properties to configure the REST component. When you define the REST configuration in the XML file, the **RestConfigurationDefinitionAutoConfiguration** property is not needed. You can disable this property as follows:

SampleCamelApplication.java

```

@SpringBootApplication(exclude = RestConfigurationDefinitionAutoConfiguration.class)
// load the spring xml file from classpath
@ImportResource("classpath:my-camel.xml")
public class SampleCamelApplication {

```

3.19.2. Graceful Shutdown

Camel now shutdowns a bit later during Spring Boot shutdown. This allows Spring Boot graceful shutdown to complete first (stopping Spring Boot HTTP server gracefully), and then afterward Camel is doing its own [Graceful Shutdown](#).

Technically **camel-spring** has changed **getPhase()** from returning **Integer.MAX_VALUE** to **Integer.MAX_VALUE - 2049**. This gives room for Spring Boot services to shut down first.

3.19.3. camel-micrometer-starter

The **uri** tags are now static instead of dynamic (by default), as potential too many tags generated due to URI with dynamic values. This can be enabled again by setting **camel.metrics.uriTagDynamic=true**.

3.19.4. camel-platform-http-starter

The **platform-http-starter** has been changed from using **camel-servlet** to use Spring HTTP server directly. Therefore, all the HTTP endpoints are no longer prefixed with the servlet context-path (default is **camel**).

For example:

```

from("platform-http:myservice")
    .to(...)

```

Then calling *myservice* would before require to include the context-path, such as <http://localhost:8080/camel/myservice>. Now the context-path is not in use, and the endpoint can be called with <http://localhost:8080/myservice>.



NOTE

The **platform-http-starter** can also be used with Rest DSL.

If the route or consumer is suspended then http status 503 is now returned instead of 404.

3.19.5. camel-twitter

The component was updated to use Twitter4j version 4.1.2, which has moved the [packages](#) used by a few of its classes. If accessing certain twitter-related data, such as the Twit status, you need to update the packages used from **twitter4j.Status** to **twitter4j.v1.Status**.

CHAPTER 4. MIGRATING TO APACHE CAMEL 3

This guide provides information on migrating from Red Hat Fuse 7 to Camel 3 on Spring Boot.

NOTE

There are important differences between Fuse 7 and Camel 3 in the components, such as modularization and XML Schema changes. See each component section for details.

docs/modules/camel-spring-boot/camel-spring-boot-migration-guide/ref-migrating-to-camel-spring-boot-3.adoc == Java versions Camel 3 supports Java 17 and Java 11 but not Java 8.

In Java 11 the JAXB modules have been **removed** from the JDK, therefore you will need to add them as Maven dependencies (if you use JAXB such as when using XML DSL or the camel-jaxb component):

```
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>

<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-core</artifactId>
  <version>2.3.0.1</version>
</dependency>

<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>2.3.2</version>
</dependency>
```

NOTE : The Java Platform, Standard Edition 11 Development Kit (JDK 11) is deprecated in Camel Spring Boot 3.x release version and not supported from the further 4.x release versions.

4.1. MODULARIZATION OF CAMEL-CORE

In Camel 3.x, **camel-core** has been split into many JARs as follows:

- camel-api
- camel-base
- camel-caffeine-lrucache
- camel-cloud
- camel-core
- camel-jaxp
- camel-main
- camel-management-api

- camel-management
- camel-support
- camel-util
- camel-util-json

Maven users of Apache Camel can keep using the dependency **camel-core** which has transitive dependencies on all of its modules, except for **camel-main**, and therefore no migration is needed.

4.2. MODULARIZATION OF COMPONENTS

In Camel 3.x, some of the camel-core components are moved into individual components.

- camel-attachments
- camel-bean
- camel-browse
- camel-controlbus
- camel-dataformat
- camel-dataset
- camel-direct
- camel-directvm
- camel-file
- camel-language
- camel-log
- camel-mock
- camel-ref
- camel-rest
- camel-saga
- camel-scheduler
- camel-seda
- camel-stub
- camel-timer
- camel-validator
- camel-vm

- camel-xpath
- camel-xslt
- camel-xslt-saxon
- camel-zip-deflater

4.3. DEFAULT SHUTDOWN STRATEGY

Red Hat build of Apache Camel supports a shutdown strategy using **org.apache.camel.spi.ShutdownStrategy** which is responsible for shutting down routes in a graceful manner. Red Hat build of Apache Camel provides a default strategy in the **org.apache.camel.impl.engine.DefaultShutdownStrategy** to handle the graceful shutdown of the routes.



NOTE

The **DefaultShutdownStrategy** class has been moved from package **org.apache.camel.impl** to **org.apache.camel.impl.engine** in Apache Camel 3.x.

When you configure a simple scheduled route policy to stop a route, the route stopping algorithm is automatically integrated with the graceful shutdown procedure. This means that the task waits until the current exchange has finished processing before shutting down the route. You can set a timeout, however, that forces the route to stop after the specified time, irrespective of whether or not the route has finished processing the exchange.

During graceful shutdown, If you enable the DEBUG logging level on **org.apache.camel.impl.engine.DefaultShutdownStrategy**, then it logs the same inflight exchange information.

```
2015-01-12 13:23:23,656 [- ShutdownTask] INFO DefaultShutdownStrategy - There are 1 inflight
exchanges:
InflightExchange: [exchangeId=ID-test-air-62213-1421065401253-0-3, fromRouteId=route1,
routeId=route1, nodeId=delay1, elapsed=2007, duration=2017]
```

If you do not want to see these logs, you can turn this off by setting the option **logInflightExchangesOnTimeout** to false.

```
context.getShutdownStrategy().setLogInflightExchangesOnTimeout(false);
```

4.4. MULTIPLE CAMELCONTEXTS PER APPLICATION NOT SUPPORTED

Support for multiple CamelContexts has been removed and only one CamelContext per deployment is recommended and supported. The **context** attribute on the various Camel annotations such as **@EndpointInject**, **@Produce**, **@Consume** etc. has therefore been removed.

4.5. DEPRECATED APIS AND COMPONENTS

All deprecated APIs and components from Camel 2.x have been removed in Camel 3.

4.5.1. Removed components

All deprecated components from Camel 2.x are removed in Camel 3.x, including the old **camel-http**, **camel-hdfs**, **camel-mina**, **camel-mongodb**, **camel-netty**, **camel-netty-http**, **camel-quartz**, **camel-restlet** and **camel-rx** components.

- Removed **camel-jibx** component.
- Removed **camel-boon** dataformat.
- Removed the **camel-linkedin** component as the LinkedIn API 1.0 is no longer [supported](#). Support for the new 2.0 API is tracked by [CAMEL-13813](#).
- The **camel-zookeeper** has its route policy functionality removed, instead use **ZooKeeperClusterService** or the **camel-zookeeper-master** component.
- The **camel-jetty** component no longer supports producer (which has been removed), use **camel-http** component instead.
- The **twitter-streaming** component has been removed as it relied on the deprecated Twitter Streaming API and is no longer functional.

4.5.2. Renamed components

Following components are renamed in Camel 3.x.

- The **Camel-microprofile-metrics** has been renamed to **camel-micrometer**
- The **test** component has been renamed to **dataset-test** and moved out of **camel-core** into **camel-dataset** JAR.
- The **http4** component has been renamed to **http**, and it's corresponding component package from **org.apache.camel.component.http4** to **org.apache.camel.component.http**. The supported schemes are now only **http** and **https**.
- The **hdfs2** component has been renamed to **hdfs**, and it's corresponding component package from **org.apache.camel.component.hdfs2** to **org.apache.camel.component.hdfs**. The supported scheme is now **hdfs**.
- The **mina2** component has been renamed to **mina**, and it's corresponding component package from **org.apache.camel.component.mina2** to **org.apache.camel.component.mina**. The supported scheme is now **mina**.
- The **mongodb3** component has been renamed to **mongodb**, and it's corresponding component package from **org.apache.camel.component.mongodb3** to **org.apache.camel.component.mongodb**. The supported scheme is now **mongodb**.
- The **netty4-http** component has been renamed to **netty-http**, and it's corresponding component package from **org.apache.camel.component.netty4.http** to **org.apache.camel.component.netty.http**. The supported scheme is now **netty-http**.
- The **netty4** component has been renamed to **netty**, and it's corresponding component package from **org.apache.camel.component.netty4** to **org.apache.camel.component.netty**. The supported scheme is now **netty**.

- The **quartz2** component has been renamed to **quartz**, and it's corresponding component package from **org.apache.camel.component.quartz2** to **org.apache.camel.component.quartz**. The supported scheme is now **quartz**.
- The **rxjava2** component has been renamed to **rxjava**, and it's corresponding component package from **org.apache.camel.component.rxjava2** to **org.apache.camel.component.rxjava**.
- Renamed **camel-jetty9** to **camel-jetty**. The supported scheme is now **jetty**.

4.6. CHANGES TO CAMEL COMPONENTS

4.6.1. Mock component

The **mock** component has been moved out of **camel-core**. Because of this a number of methods on its *assertion clause builder* are removed.

4.6.2. ActiveMQ

If you are using the **activemq-camel** component, then you should migrate to use **camel-activemq** component, where the component name has changed from **org.apache.activemq.camel.component.ActiveMQComponent** to **org.apache.camel.component.activemq.ActiveMQComponent**.

4.6.3. AWS

The component **camel-aws** has been split into multiple components:

- camel-aws-cw
- camel-aws-ddb (which contains both ddb and ddbstreams components)
- camel-aws-ec2
- camel-aws-iam
- camel-aws-kinesis (which contains both kinesis and kinesis-firehose components)
- camel-aws-kms
- camel-aws-lambda
- camel-aws-mq
- camel-aws-s3
- camel-aws-sdb
- camel-aws-ses
- camel-aws-sns
- camel-aws-sqs
- camel-aws-swf

**NOTE**

It is recommended to add specific dependencies for these components.

4.6.4. Camel CXF

The **camel-cxf** JAR has been divided into SOAP vs REST and Spring and non Spring JARs. It is recommended to choose the specific JAR from the following list when migrating from **came-cxf**.

- **camel-cxf-soap**
- **camel-cxf-spring-soap**
- **camel-cxf-rest**
- **camel-cxf-spring-rest**
- **camel-cxf-transport**
- **camel-cxf-spring-transport**

For example, if you were using CXF for SOAP and with Spring XML, then select **camel-cxf-spring-soap** and **camel-cxf-spring-transport** when migrating from **camel-cxf**.

When using Spring Boot, choose from the following starter when you migrate from **camel-cxf-starter** to SOAP or REST:

- **camel-cxf-soap-starter**
- **camel-cxf-rest-starter**

4.6.4.1. Camel CXF changed namespaces

The **camel-cxf** XML XSD schemas has also changed namespaces.

Table 4.1. Changes to namespaces

Old Namespace	New Namespace
http://camel.apache.org/schema/cxf	http://camel.apache.org/schema/cxf/jaxws
http://camel.apache.org/schema/cxf/camel-cxf.xsd	http://camel.apache.org/schema/cxf/jaxws/camel-cxf.xsd
http://camel.apache.org/schema/cxf	http://camel.apache.org/schema/cxf/jaxrs
http://camel.apache.org/schema/cxf/camel-cxf.xsd	http://camel.apache.org/schema/cxf/jaxrs/camel-cxf.xsd

The **camel-cxf** SOAP component is moved to a new **jaxws** sub-package, that is, **org.apache.camel.component.cxf** is now **org.apache.camel.component.cxf.jaxws**. For example, the **CxfComponent** class is now located in **org.apache.camel.component.cxf.jaxws**.

4.6.5. FHIR

The camel-fhir component has upgraded it's hapi-fhir dependency to 4.1.0. The default FHIR version has been changed to R4. Therefore, if DSTU3 is desired it has to be explicitly set.

4.6.6. Kafka

The **camel-kafka** component has removed the options **bridgeEndpoint** and **circularTopicDetection** as this is no longer needed as the component is acting as bridging would work on Camel 2.x. In other words **camel-kafka** will send messages to the topic from the endpoint uri. To override this use the **KafkaConstants.OVERRIDE_TOPIC** header with the new topic. See more details in the **camel-kafka** component documentation.

4.6.7. Telegram

The **camel-telegram** component has moved the authorization token from uri-path to a query parameter instead, e.g. migrate

```
telegram:bots/myTokenHere
```

to

```
telegram:bots?authorizationToken=myTokenHere
```

4.6.8. JMX

If you run Camel standalone with just **camel-core** as a dependency, and you want JMX enabled out of the box, then you need to add **camel-management** as a dependency.

For using **ManagedCamelContext** you now need to get this extension from **CamelContext** as follows:

```
ManagedCamelContext managed = camelContext.getExtension(ManagedCamelContext.class);
```

4.6.9. XSLT

The XSLT component has moved out of camel-core into **camel-xslt** and **camel-xslt-saxon**. The component is separated so **camel-xslt** is for using the JDK XSTL engine (Xalan), and **camel-xslt-saxon** is when you use Saxon. This means that you should use **xslt** and **xslt-saxon** as component name in your Camel endpoint URIs. If you are using XSLT aggregation strategy, then use **org.apache.camel.component.xslt.saxon.XsltSaxonAggregationStrategy** for Saxon support. And use **org.apache.camel.component.xslt.saxon.XsltSaxonBuilder** for Saxon support if using xslt builder. Also notice that **allowStax** is also only supported in **camel-xslt-saxon** as this is not supported by the JDK XSLT.

4.6.10. XML DSL Migration

The XML DSL has been changed slightly.

The custom load balancer EIP has changed from **<custom>** to **<customLoadBalancer>**

The XMLSecurity data format has renamed the attribute **keyOrTrustStoreParametersId** to **keyOrTrustStoreParametersRef** in the **<secureXML>** tag.

The `<zipFile>` data format has been renamed to `<zipfile>`.

4.7. MIGRATING CAMEL MAVEN PLUGINS

The **camel-maven-plugin** has been split up into two maven plugins:

camel-maven-plugin

camel-maven-plugin has the **run** goal, which is intended for quickly running Camel applications standalone. See <https://camel.apache.org/manual/camel-maven-plugin.html> for more information.

camel-report-maven-plugin

The **camel-report-maven-plugin** has the **validate** and **route-coverage** goals which is used for generating reports of your Camel projects such as validating Camel endpoint URLs and route coverage reports, etc. See <https://camel.apache.org/manual/camel-report-maven-plugin.html> for more information.

CHAPTER 5. UPGRADING RED HAT BUILD OF APACHE CAMEL ON SPRING BOOT VERSION WITH CAMEL UPDATE RECIPES

Migrating the code for Apache Camel on Spring Boot often involves adapting to new Camel API changes and renaming the classes. To address this, you can use Camel update recipes based on OpenRewrite. These recipes assist with manual migrations and make it more efficient.

You can update the Red Hat build of Apache Camel on Spring Boot application using Camel JBang. Camel JBang provides the **camel update** command. It has two main operations:

- **run**: Executes the actual update process

The update process uses the [Apache Camel Open Rewrite recipes](#). It supports following application types:

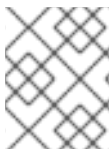
- Plain Camel (camel-main)
- Camel Quarkus
- Camel Spring Boot

Here, Camel and Camel Spring Boot updates mainly use the **camel-upgrade-recipes**, while Camel Quarkus updates involve both the Quarkus runtime (via [Rewrite Quarkus](#)) and Apache Camel recipes.

5.1. RUNNING CAMEL UPDATES

To perform the update to this version, use:

```
$ camel update run {camelSpringBootVersion} --runtime=spring-boot
```



NOTE

The update commands must be executed in the project directory that contains the **pom.xml** file.

5.1.1. Configuration Options

You can customize the update process with several options that are available:

- **--runtime**: Specifies the application type:
 - **spring-boot** - Camel Spring Boot applications
- **--repos**: Additional Maven repositories to use during the update
- **--dry-run**: Preview the changes without applying them
- **--extraActiveRecipes**: Comma-separated list of additional recipe names to apply
- **--extraRecipeArtifactCoordinates**: Comma-separated list of Maven coordinates for extra recipes (format: groupId:artifactId:version)

- **--help** : To see all available options.

5.1.2. Updating a plain Camel application

Following example shows how to update a plain Camel application.

```
$ camel update run {CamelUpgradeRecipesVersion} --runtime=camel-main --  
repos=https://myMaven/repo --extraActiveRecipes=my.first.Recipe,my.second.Recipe --  
extraRecipeArtifactCoordinates=ex.my.org:recipes:1.0.0
```

5.1.3. Updating a Spring Boot application

You can use the following command to update a Spring Boot application. You can use the - **extraActiveRecipes** option to run the extra Spring Boot upgrade.

```
$ camel update run {CamelUpgradeRecipesVersion} --runtime=spring-boot --  
extraActiveRecipes=org.openrewrite.java.spring.boot3.UpgradeSpringBoot_3_3 --  
extraRecipeArtifactCoordinates=org.openrewrite.recipe:rewrite-spring:6.0.2
```